

`-l` -- Low quality (480p, fast rendering)

Quality options are:

`-ql` -- 480p (development)

`-qm` -- 720p (standard)

`-qh` -- 1080p (production)

`-qk` -- 4K (high-end)

Essential positioning is as follows:

- `to_edge(UP/DOWN/LEFT/RIGHT)` -- Align to screen edges
- `next_to(object, direction)` -- Position relative to other objects
- `shift(vector)` -- Move by specified amount
- `move_to(position)` -- Move to exact coordinates

Coordinate system is:

- *Origin* (0,0) at screen centre
- *Positive X* extends right
- *Positive Y* extends up
- *Standard vectors*: UP, DOWN, LEFT, RIGHT, ORIGIN

Algorithm and data structure visualisation

For horizontal distance, use the following code:

```
from manim import *

class BraceAnnotation(Scene):
    def construct(self):
        dot = Dot([-2, -1, 0])
        dot2 = Dot([2, 1, 0])
        line = Line(dot.get_center(), dot2.get_center()).
set_color(ORANGE)
        b1 = Brace(line)
        b1text = b1.get_text("Horizontal distance")
```

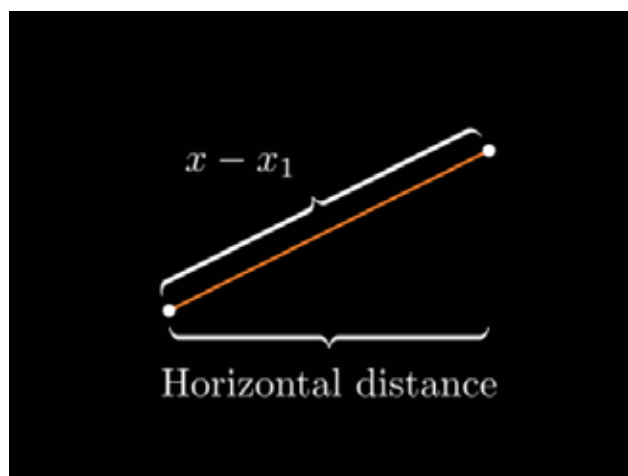


Figure 3: Horizontal distance

```
b2 = Brace(line, direction=line.copy().rotate(PI /
2).get_unit_vector())
b2text = b2.get_text("x-x_1")
self.add(line, dot, dot2, b1, b2, b1text, b2text)
```

You can refer to Figure 3 for the output of the above code.

For vector arrow, use:

```
from manim import *

class VectorArrow(Scene):
    def construct(self):
        dot = Dot(ORIGIN)
        arrow = Arrow(ORIGIN, [2, 2, 0], buff=0)
        numberplane = NumberPlane()
        origin_text = Text('(0, 0)').next_to(dot, DOWN)
        tip_text = Text('(2, 2)').next_to(arrow.get_end(),
RIGHT)
        self.add(numberplane, dot, arrow, origin_text, tip_
text)
```

The output of the above code is given in Figure 4.

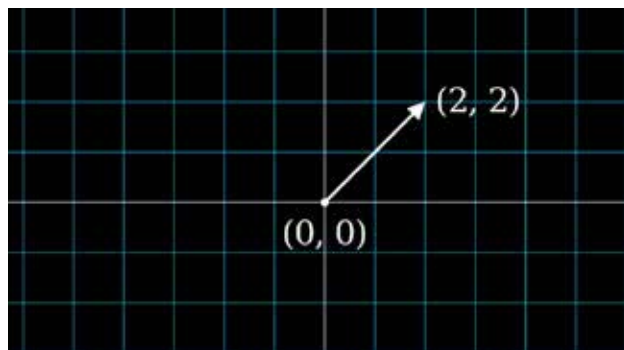


Figure 4: Vector arrow

The code for Boolean operations is:

```
from manim import *

class BooleanOperations(Scene):
    def construct(self):
        ellipse1 = Ellipse(
            width=4.0, height=5.0, fill_opacity=0.5,
color=BLUE, stroke_width=10
        ).move_to(LEFT)
        ellipse2 = ellipse1.copy().set_color(color=RED).
move_to(RIGHT)
        bool_ops_text = MarkupText("<u>Boolean Operation</
u>").next_to(ellipse1, UP * 3)
        ellipse_group = Group(bool_ops_text, ellipse1,
```